Experiment 1: Implementation of Linear and Logistic Regression

---

1. Dataset Source

- **Logistic Regression Dataset:**
  Diabetic Retinopathy Debrecen Dataset
  Link: https://archive.ics.uci.edu/dataset/329/diabetic+retinopathy+debrecen
- **Linear Regression Dataset:**
  Pima Indians Diabetes Dataset
  Link: https://www.kaggle.com/datasets/uciml/pima-indians-diabetes-database

---

2. Dataset Description

Logistic Regression Dataset (Retinopathy)

This dataset contains features extracted from retinal images to detect diabetic retinopathy.

Features:

- 19 numerical features (e.g., microaneurysms, exudates)
- Automatically extracted from medical images

Target:

- `Class`
  - 0 → No Diabetic Retinopathy
  - 1 → Presence of Diabetic Retinopathy

Size:

- 1151 instances
- 20 attributes (19 features + 1 class)

Characteristics:

- Preprocessed dataset (no raw images)
- Suitable for classification tasks

Linear Regression Dataset (Diabetes)

This dataset contains medical records for predicting diabetes-related measurements.

Features:

- Pregnancies, BMI, Age, Insulin, etc.

Target:

- `Glucose` (continuous variable)

Size:

- 768 instances
- 9 attributes

Characteristics:

- Real-world healthcare dataset
- Used for both regression & classification

---

3. Mathematical Formulation

Logistic Regression

The hypothesis function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Where:

$$z = w_0 + w_1 x_1 + w_2 x_2 + ... + w_n x_n$$

Decision boundary:

$$y = \begin{cases} 1 & \text{if } \sigma(z) \geq 0.5 \\ 0 & \text{otherwise} \end{cases}$$

Linear Regression

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n$$

Loss Function:

$$MSE = \frac{1}{n} \sum (y_{actual} - y_{predicted})^2$$

---

4. Algorithm Limitations

Logistic Regression:

- Cannot model complex non-linear patterns
- Sensitive to feature scaling
- Assumes linear separability
- Performance drops with highly correlated features

Linear Regression:

- Assumes linear relationship
- Sensitive to outliers
- Affected by multicollinearity

- Not suitable for categorical targets

---

5. Methodology / Workflow

Steps Followed:

Logistic Regression (Retinopathy):

1. Load `.arff` dataset
2. Convert byte values to integer
3. Separate features and target
4. Apply feature scaling
5. Train Logistic Regression model
6. Predict and evaluate

Linear Regression (Diabetes):

1. Load CSV dataset
2. Select `Glucose` as target
3. Perform train-test split
4. Apply feature scaling
5. Train Linear Regression model
6. Predict and evaluate

Workflow Diagram

Data Collection → Preprocessing → Train-Test Split → Model Training → Prediction → Evaluation

Implementation :

[2]

```python
import pandas as pd
from scipy.io import arff

data, meta = arff.loadarff('messidor_features.arff')
df = pd.DataFrame(data)
df.head()
```

4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	Class
2.0	19.0	18.0	14.0	49.895756	17.775994	5.270920	0.771761	0.018632	0.006864	0.003923	0.003923	0.486903	0.100025	1.0	b'0'
2.0	18.0	16.0	13.0	57.709936	23.799994	3.325423	0.234185	0.003903	0.003903	0.003903	0.003903	0.520908	0.144414	0.0	b'0'
9.0	54.0	47.0	33.0	55.831441	27.993933	12.687485	4.852282	1.393889	0.373252	0.041817	0.007744	0.530904	0.128548	0.0	b'1'
3.0	50.0	43.0	31.0	40.467228	18.445954	9.118901	3.079428	0.840261	0.272434	0.007653	0.001531	0.483284	0.114790	0.0	b'0'
4.0	41.0	39.0	27.0	18.026254	8.570709	0.410381	0.000000	0.000000	0.000000	0.000000	0.000000	0.475935	0.123572	0.0	b'1'

[3]

```python
#fix target column
df['Class'] = df['Class'].str.decode('utf-8').astype(int)
df.head()
```

4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	Class
2.0	19.0	18.0	14.0	49.895756	17.775994	5.270920	0.771761	0.018632	0.006864	0.003923	0.003923	0.486903	0.100025	1.0	0
2.0	18.0	16.0	13.0	57.709936	23.799994	3.325423	0.234185	0.003903	0.003903	0.003903	0.003903	0.520908	0.144414	0.0	0
9.0	54.0	47.0	33.0	55.831441	27.993933	12.687485	4.852282	1.393889	0.373252	0.041817	0.007744	0.530904	0.128548	0.0	1
3.0	50.0	43.0	31.0	40.467228	18.445954	9.118901	3.079428	0.840261	0.272434	0.007653	0.001531	0.483284	0.114790	0.0	0
4.0	41.0	39.0	27.0	18.026254	8.570709	0.410381	0.000000	0.000000	0.000000	0.000000	0.000000	0.475935	0.123572	0.0	1

[4]

```python
x = df.drop('Class', axis=1)
y = df['Class']
```

[6]

```python
from sklearn.model_selection import train_test_split
X_train , X_test , y_train , y_test = train_test_split(
    x , y , test_size=0.2 , random_state=42
)
```

[7]

```python
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

[8]

```python
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=2000)
model.fit(X_train, y_train)
```

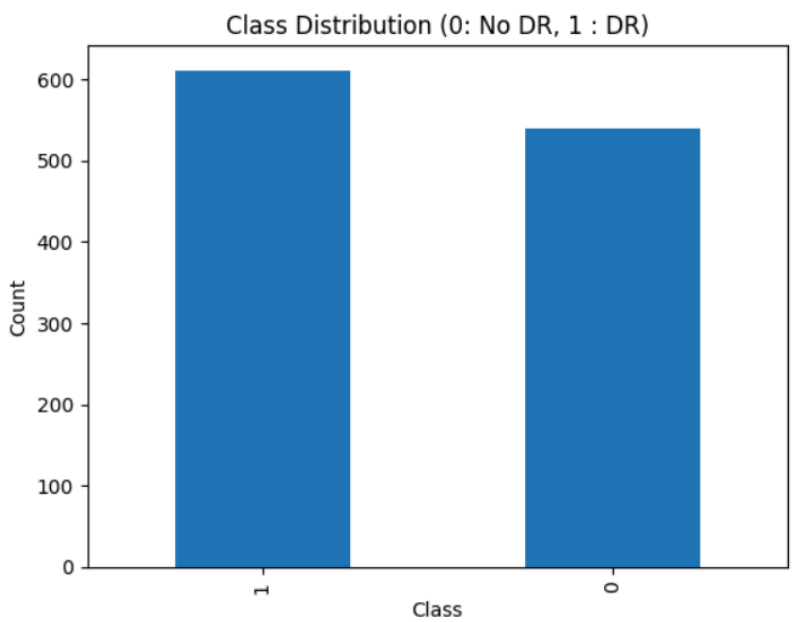
```
LogisticRegression
LogisticRegression(max_iter=2000)
```

[10]
✓ 0s

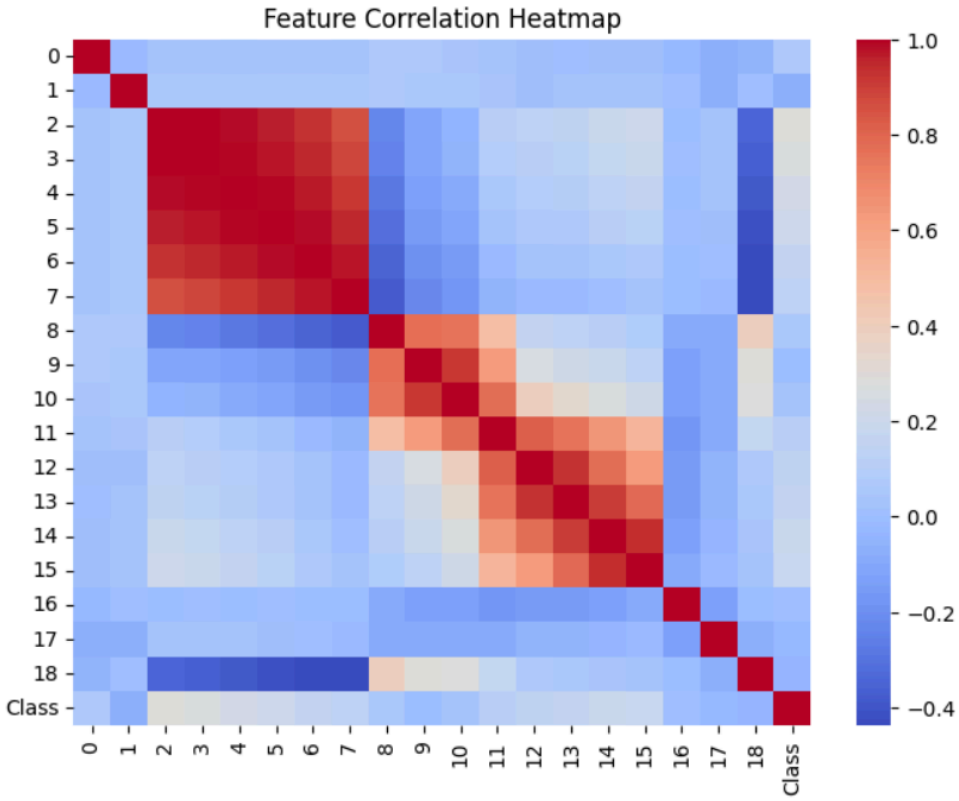
```
from sklearn.metrics import accuracy_score, confusion_matrix

y_pred = model.predict(X_test)
print('Accuracy:', accuracy_score(y_test,y_pred))
print('Confusion Matrix:\n', confusion_matrix(y_test,y_pred))
```

```
Accuracy: 0.70995670995671
Confusion Matrix:
 [[83 20]
 [47 81]]
```

```
import matplotlib.pyplot as plt

df['Class'].value_counts().plot(kind='bar')
plt.title('Class Distribution (0: No DR, 1 : DR)')
plt.xlabel('Class')
plt.ylabel('Count')
plt.show()
```




```
import seaborn as sns

plt.figure(figsize=(8,6))
sns.heatmap(df.corr(), cmap='coolwarm')
plt.title('Feature Correlation Heatmap')
plt.show()
```


Feature Correlation Heatmap
0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
Class
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
−0.4

```python
import pandas as pd

df = pd.read_csv('diabetes.csv')
print(df.head())
print(df.info())
print(df.describe())
```

```
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0            6      148             72             35        0  33.6
1            1       85             66             29        0  26.6
2            8      183             64              0        0  23.3
3            1       89             66             23       94  28.1
4            0      137             40             35      168  43.1

   DiabetesPedigreeFunction  Age  Outcome
0                     0.627   50        1
1                     0.351   31        0
2                     0.672   32        1
3                     0.167   21        0
4                     2.288   33        1
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Pregnancies               768 non-null    int64
 1   Glucose                   768 non-null    int64
 2   BloodPressure             768 non-null    int64
 3   SkinThickness             768 non-null    int64
 4   Insulin                   768 non-null    int64
 5   BMI                       768 non-null    float64
```

[4] ✓ 0s

```python
x = df.drop('Glucose', axis=1)
y = df['Glucose']
```

[5] ✓ 0s

```python
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    x , y , test_size=0.2 , random_state=42
)
```

[6] ✓ 0s

```python
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

[7] ✓ 0s

```python
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_train, y_train)
```

```
▸ LinearRegression
```

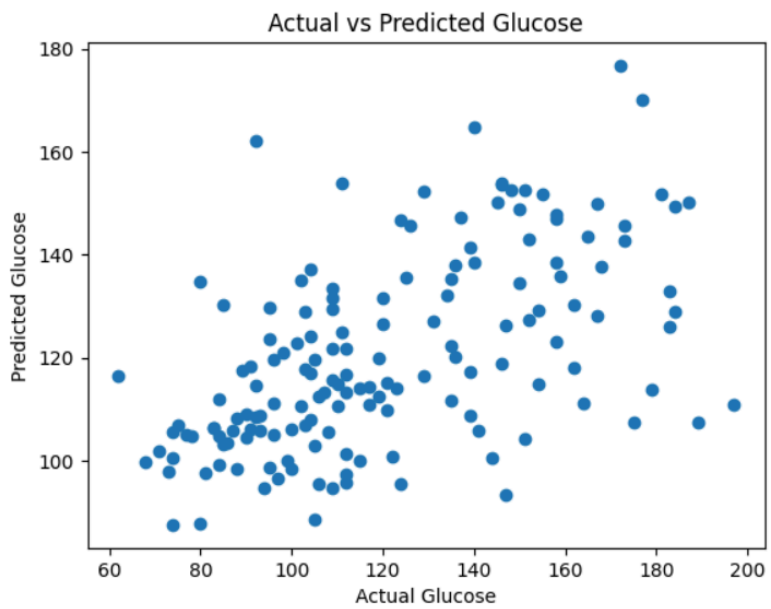
[9]
0s

```
from sklearn.metrics import mean_squared_error, r2_score

y_pred = model.predict(X_test)

print("MSE: ", mean_squared_error(y_test, y_pred))
print("R2: ", r2_score(y_test, y_pred))
```

```
MSE:  704.6762254136174
R2:  0.299741697015366
```

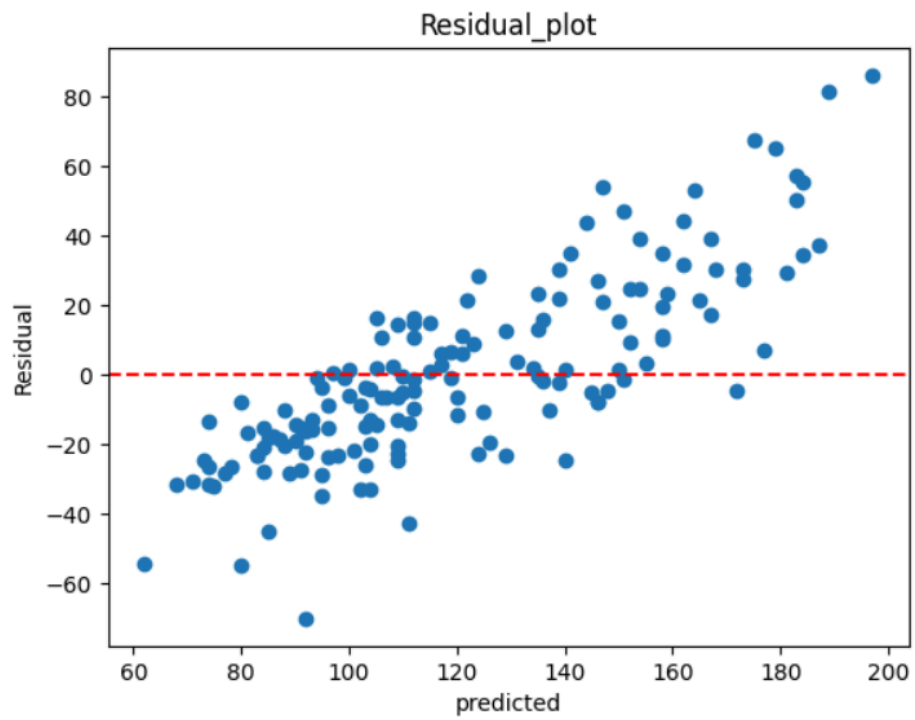
[10]
0s

```
import matplotlib.pyplot as plt

plt.scatter(y_test, y_pred)
plt.xlabel('Actual Glucose')
plt.ylabel('Predicted Glucose')
plt.title('Actual vs Predicted Glucose')
plt.show()
```

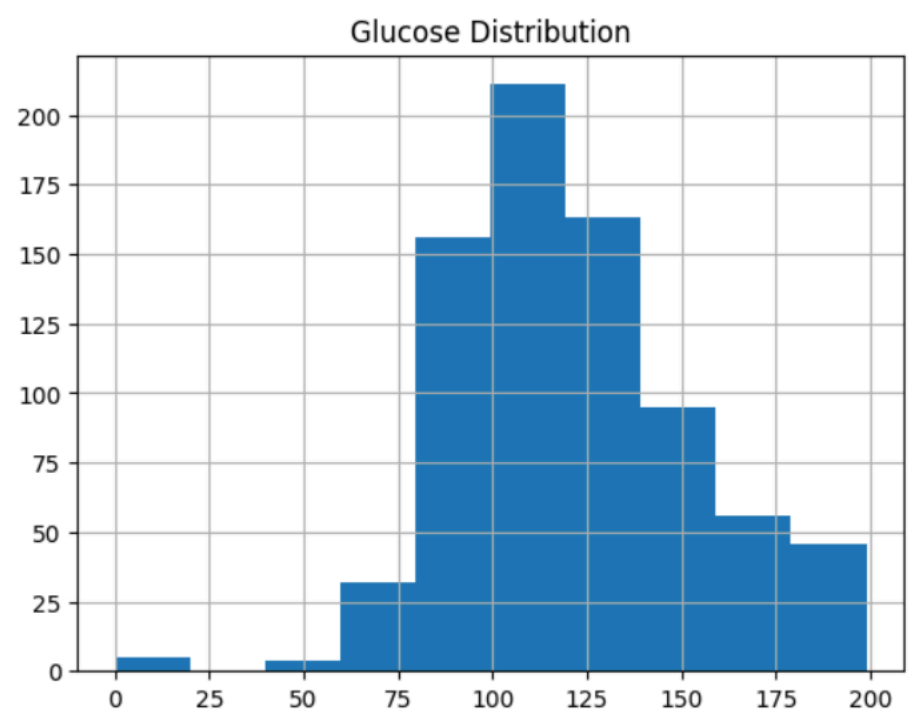


[11]
0s

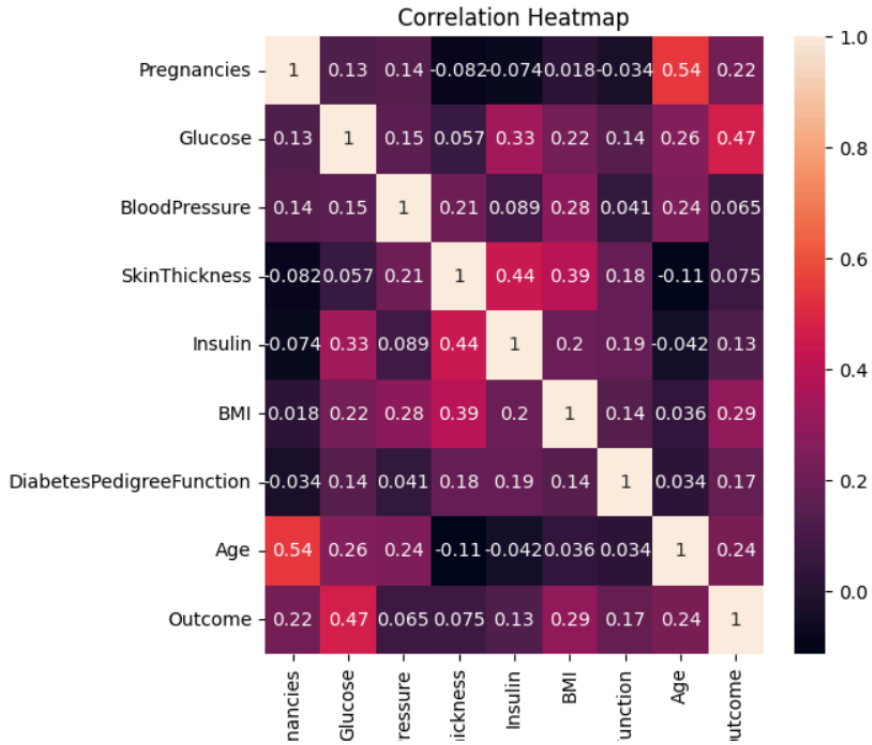
```
residual = y_test - y_pred
plt.scatter(y_test, residual)
plt.axhline(y=0, color='r', linestyle='--')
plt.xlabel('predicted')
plt.ylabel('Residual')
plt.title('Residual_plot')
plt.show()
```

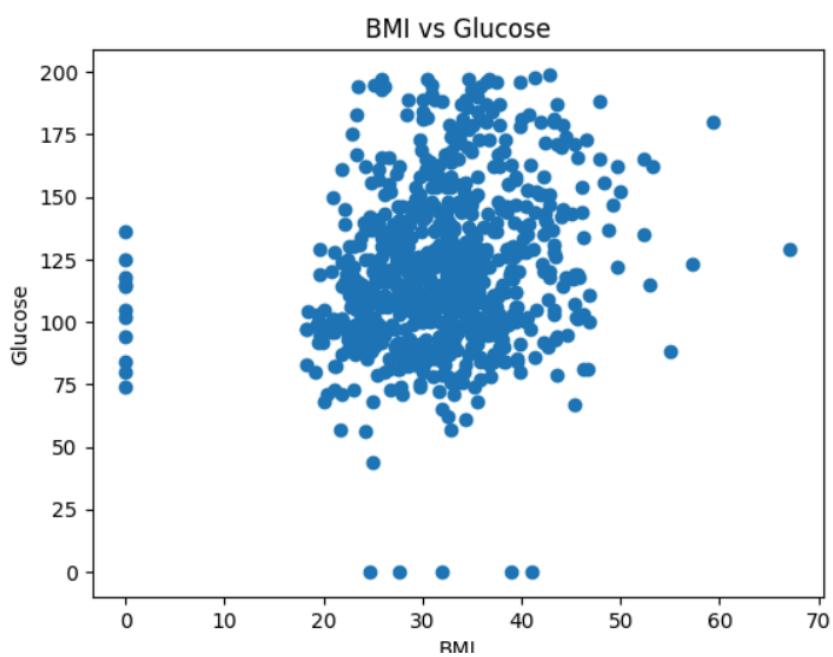



```
df['Glucose'].hist()
plt.title('Glucose Distribution')
plt.show()
```



```
import seaborn as sns

plt.figure(figsize=(6,6))
sns.heatmap(df.corr(), annot=True)
plt.title("Correlation Heatmap")
plt.show()
```





---

6. Performance Analysis

Logistic Regression:

- Accuracy: ~70–75% (approx.)
- Evaluation tools:
  - Confusion Matrix
  - ROC Curve

Interpretation:

- Model identifies retinopathy cases reasonably well
- Some misclassification due to overlapping features

Linear Regression:

- Metrics:
  - Mean Squared Error (MSE)
  - R^2 Score

Interpretation:

- Moderate prediction accuracy
- Glucose level influenced by multiple factors

---

7. Hyperparameter Tuning

Logistic Regression:

Tuned parameters:

- `C` (regularization strength)

- `solver`

```
from sklearn.model_selection import GridSearchCV
from sklearn.linear_model import LogisticRegression

params = {
    'C': [0.01, 0.1, 1, 10],
    'solver': ['liblinear', 'lbfgs']
}

grid = GridSearchCV(LogisticRegression(max_iter=1000), params, cv=5)
grid.fit(X_train, y_train)

print(grid.best_params_)
```

Impact:

- Improved accuracy
- Better generalization

Linear Regression:

- No major hyperparameters
- Performance improved using:
    - Feature scaling
    - Feature selection
- Optional : Ridge / Lasso Regression

Conclusion

- Logistic Regression effectively classified diabetic retinopathy cases
- Linear Regression successfully predicted glucose levels
- Different datasets were used appropriately based on problem type
- Model performance depends on preprocessing and tuning